

■ Solve

`Solve[eqns, vars]` attempts to solve an equation or set of equations for the variables *vars*.

`Solve[eqns, vars, elims]` attempts to solve the equations for *vars*, eliminating the variables *elim*s.

Equations are given in the form *lhs* == *rhs*. ■ Simultaneous equations can be combined either in a list or with `&&`. ■ A single variable or a list of variables can be specified. ■ `Solve[eqns]` try to solve for all variables in *eqns*. ■ Example: `Solve[3 x + 9 == 0, x]`. ■ `Solve` gives explicit solutions as rules of the form *x* -> *sol*. ■ When there are several variables, the solution is given as a list of rules: {*x* -> *s_x*, *y* -> *s_y*, ...}. ■ When there are several solutions, `Solve` gives a list of them. ■ When a particular root has multiplicity greater than one, `Solve` gives several copies of the corresponding solution. ■ `Solve` deals primarily with linear and polynomial equations. ■ `Solve` gives generic solutions only. It discards solutions that are valid only when the parameters satisfy special conditions. `Reduce` gives the complete set of solutions. ■ `Solve` will not always be able to get explicit solutions to equations. It will give the explicit solutions it can, then give a symbolic representation of the remaining solutions, in terms of the function `Roots`. If there are sufficiently few symbolic parameters, you can then use `N` to get numerical approximations to the solutions. ■ `Solve` gives {} if there are no possible solutions to the equations. ■ `Solve[eqns, ..., Mode->Modular]` solves equations with equality required only modulo an integer. You can specify a particular modulus to use by including the equation `Modulus==p`. If you do not include such an equation, `Solve` will attempt to solve for the possible moduli. ■ See page 400. ■ See also: `Reduce`, `Eliminate`, `Roots`, `NRoots`, `FindRoot`, `LinearSolve`.